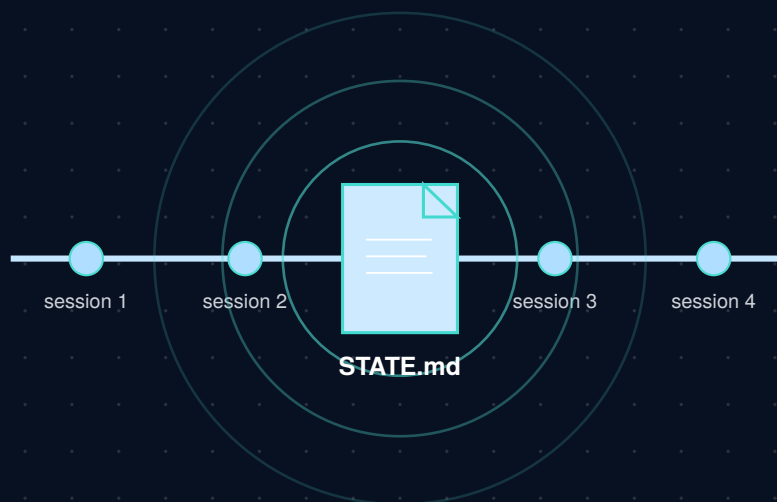


CONTINUITY

Repository-native state-as-code for persistent AI coding agents

BITwiki Research • Systems paper • June 2026



The runtime ends. The state remains.

Current truth, tasks, lessons, and evidence survive as versioned files.

Alejandro Capote • BITwiki

github.com/bitwikiorg/continuity

Abstract. Coding agents lose context at every session boundary. They may remember the current file, yet forget the architecture decision made two hours earlier, the test that already failed, or the constraint that makes an apparently simple edit dangerous. CONTINUITY addresses that gap with a small repository-native state-as-code grammar: manifest files for boot logic, state files for current truth, task files for the active queue, memory files for durable lessons, append-only history, and artifact directories for evidence. The system is deliberately narrow. It is not an agent runtime, a vector database, or a general retrieval-augmented generation layer. It is a file convention for keeping high-signal agent state close to the code.

The argument is two-sided. Work on external memory for large language models supports persistent state outside the context window. Current coding tools also confirm the practical value of repository-local instruction files such as **AGENTS.md** and **CLAUDE.md**. At the same time, recent empirical studies show that repository context files can raise cost and reduce task success when they are bloated, stale, or internally inconsistent. The useful system is therefore small, scoped, current, and validated. This paper formalizes that stance, defines the core artifact classes, models state loading and reconciliation, extends the design to multi-agent collaboration, and outlines an evaluation and threat model suitable for an arXiv-level systems paper.

Keywords. AI coding agents; persistent state; repository-local context; **AGENTS.md**; software architecture knowledge; workflow provenance; multi-agent coordination; prompt injection.

1 Introduction

Coding agents now work at repository scale. They inspect code, edit files, run tests, collect logs, and synthesize patches across many steps. That workflow exposes a recurring weakness: each session begins with partial amnesia. The next agent may inherit the codebase, but not the local truth that made the last session coherent.

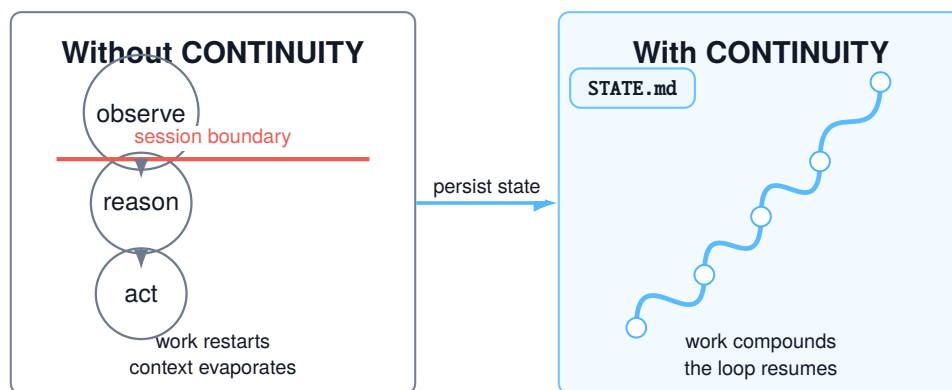


Figure 1. A session can end without erasing the work state. That is the core operational promise.

Four kinds of state are commonly lost. First, *current truth* drifts: the repository may have changed, but the agent still acts on an earlier plan. Second, *attempt history* disappears: failures are repeated because earlier experiments were never captured. Third, *operational constraints* get buried in chat logs, issue threads, or a

developer’s head. Fourth, *handoff state* is weak: one agent cannot easily tell another what remains, what is blocked, and what evidence already exists.

Tool practice has started to converge on a partial fix. Codex loads **AGENTS.md** before work and composes instructions across global, repository, and directory scopes [1]. Claude Code documents persistent repository context through **CLAUDE.md** and distinguishes that soft context from hard enforcement [2, 3]. Hermes progressively discovers local context files as the agent moves through a project [4]. OpenClaw stores memory as plain Markdown on disk rather than hidden platform state [5]. These systems already treat the repository as part of the agent interface.

CONTINUITY extends that interface into a compact state substrate. It does not ask one file to do everything. It separates stable boot guidance from mutable state, active tasks, learned lessons, append-only history, and generated evidence. The minimal core stays intentionally small:

Minimal CONTINUITY core

AGENTS.md	boot order, governance, load rules
STATE.md	current project truth
TODO.md	executable task queue
LEARNINGS.md	compact reflection and reusable lessons
MEMORY.md	durable facts index, loaded selectively
completed/	append-only records of finished work
snapshots/	recovery checkpoints
artifacts/	outputs, logs, test evidence
CHECKPOINT.md	approval gates for high-risk actions

The point is not to create a repository diary. The point is to keep the smallest set of files that preserve continuity across sessions and across agents.

Recent evidence makes the design constraint sharper. Gloaguen et al. find that repository context files do not reliably improve coding-agent task success and can increase inference cost by more than 20% on average [6]. Dos Santos et al. report six recurring configuration smells in **AGENTS.md**-style files, including *context bloat* (too much irrelevant or diffuse instruction), *lint leakage* (tool-output noise or rules pasted directly into the manifest), *skill leakage* (specialized or personal instructions spilling into shared context), and *conflicting instructions* [7]. Those results rule out the obvious mistake: more prose is not automatically better state.

The rest of the paper develops the positive version of that negative lesson. CONTINUITY works when it remains compact, scoped, current, and reviewable. The next sections show where that stance comes from, how the file grammar works, how multi-agent collaboration fits, and how the design should be evaluated and constrained.

2 Foundations

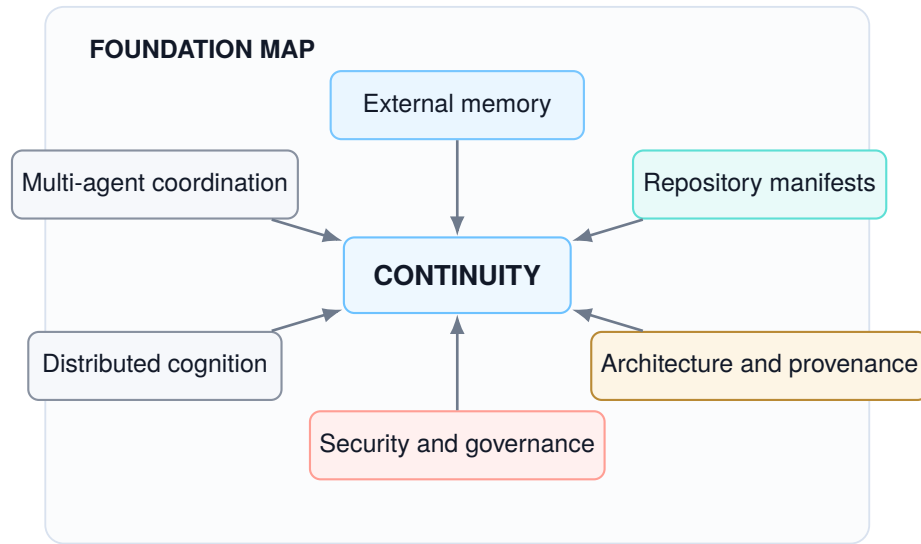


Figure 2. Six literatures support the proposal. The paper sits at their intersection rather than inside only one of them.

2.1 External memory for coding agents

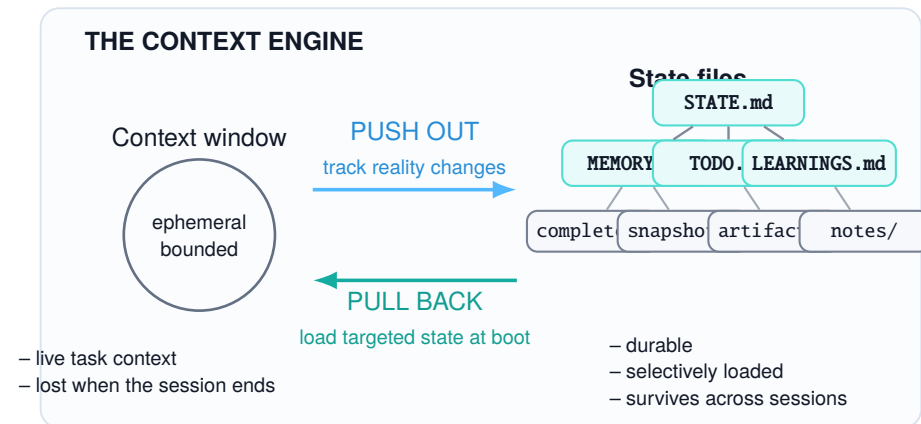


Figure 3. Durable state stays outside the context window. The model only pulls back the smallest useful slice.

Modern large language model systems are powerful precisely where their memory is weak. The context window is large but still finite, and tool-use traces quickly crowd out older information. That is why the memory literature repeatedly pushes state outside the model. MemGPT treats memory as a tiered system that shuttles information between active context and longer-term stores [8]. MemoryBank studies selective retention and memory updates over time [9]. Generative Agents and Reflexion both show that reusable behavior emerges when experience is written down, revisited, and distilled into higher-value summaries [10, 11]. Survey work reaches the same conclusion: durable agent memory needs explicit write, manage, and read loops [12].

CONTINUITY adopts that external-memory stance but narrows the medium. It uses repository files as the persistence layer because coding agents and developers already live there.

2.2 Repository-local manifests are already standard practice

The repository has become part of the prompt surface. Codex, Claude Code, Hermes, and OpenClaw all expose repository-local instruction or memory files [1, 2, 4, 5]. CONTINUITY does not need to justify that starting point. What it adds is separation of concerns. A manifest should not double as a task log; current state should not double as an archive; and evidence should not hide inside prose.

2.3 Architecture knowledge and workflow provenance

Software repositories do not store code alone. They also store design intent, constraints, decisions, tradeoffs, and evidence. Classical architecture work treats those as first-class parts of the system description [13–17]. Empirical studies show that architecture knowledge often ends up scattered through ordinary development artifacts, including issue trackers [18]. Reproducibility and provenance work make the same demand from another angle: important work should be reconstructable from inputs, actions, outputs, and version history [19–22].

CONTINUITY places a lightweight state layer on top of those ideas. **STATE.md** captures what is true now. `completed/` and `artifacts/` capture what happened and what proves it.

2.4 Coordination depends on shared artifacts

Multi-agent work needs more than messaging. It needs a shared place where partial results, task claims, and review outcomes can be seen by others. Blackboard systems, the Contract Net Protocol, Linda tuple spaces, and the actor model all offer variants of that idea [23–26]. Recent multi-agent large language model systems make the same move through role separation and explicit task exchange [27–29]. computer-supported cooperative work and distributed cognition explain why this works: coordination improves when knowledge is externalized into inspectable artifacts [30–33].

CONTINUITY uses the repository as that shared surface. It is a blackboard by convention, not by runtime.

2.5 Security limits the whole design

Every file that enters context becomes part of the instruction boundary. Indirect prompt injection, prompt theft, and over-permissioned agent workflows all exploit that boundary [34–37]. CONTINUITY therefore treats Markdown as a soft interface. The enforcement layer lives elsewhere: hooks, schemas, permissions, tests, and review.

3 Artifact Grammar

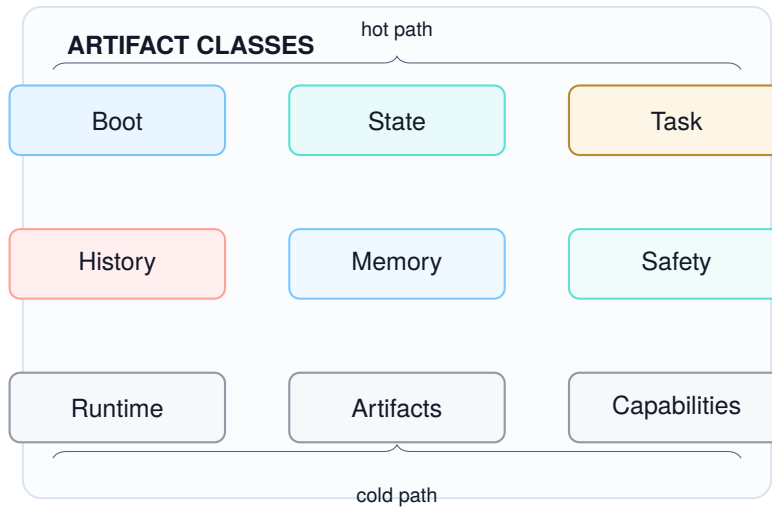


Figure 4. Classes are distinguished by how they load and how they change, not only by topic.

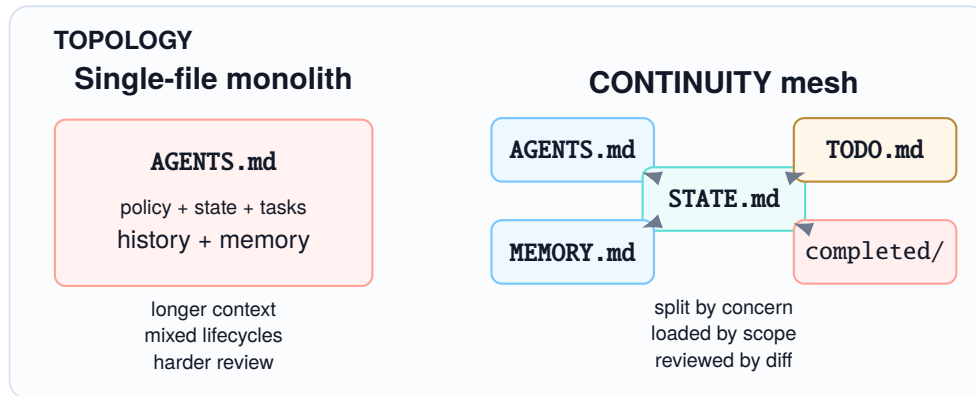


Figure 5. The mesh preserves separate lifecycles instead of forcing all agent context into one manifest.

CONTINUITY organizes repository artifacts into classes with different mutability and load rules. That distinction matters. When current state, historical notes, and reusable knowledge collapse into the same file, the result is usually drift or bloat. In this paper, upper-case filenames such as **STATE.md** denote the live state layer, while lower-case paths such as `completed/` and `artifacts/` denote history or evidence stores.

Table 1. CONTINUITY artifact classes. The key distinction is between what loads by default and what stays cold until needed.

Class	Examples	Mutability	Default load	Function
Boot	AGENTS.md , BOOT.md	rare edits	yes	startup logic, norms, governance
State	STATE.md , CONTEXT.md	overwrite	scoped	current project truth
Task	TODO.md , TASKS.md	frequent	scoped	executable queue and ownership
Memory	MEMORY.md , LEARNINGS.md	curated	selective	durable facts and distilled lessons
History	completed/, snapshots/	append-only	no	evidence and prior states
Safety	CHECKPOINT.md , WARNING.md	stable	yes	approvals and hazard notes
Runtime	claims/, runs/	structured	per run	activity trace and coordination
Artifacts	artifacts/, reviews/	append or curated	no	logs, plots, outputs, review records

3.1 Core terms

Repository-native state

Project state stored inside the repository rather than inside hidden platform memory.

Current truth The smallest accurate description of what is true right now.

State reconciliation

Updating state artifacts after code, tests, or decisions change.

Scoped loading

Resolving only the manifest, state, and task files relevant to the current path.

Semantic history

Append-only records organized by meaning: completed work, reviews, evidence, and snapshots.

Soft constraint

A model-visible instruction that is not technically enforced.

Hard constraint

A rule enforced by hooks, tests, permissions, schemas, or human approval.

3.2 Minimal core versus template library

The paper defends a *minimal reference core*, not the largest possible template set. A repository that only needs **AGENTS.md**, **STATE.md**, and **TODO.md** should not be pressured into creating ten more files. The extended topology is valuable as a library of patterns. It is not a proof that every pattern should be loaded.

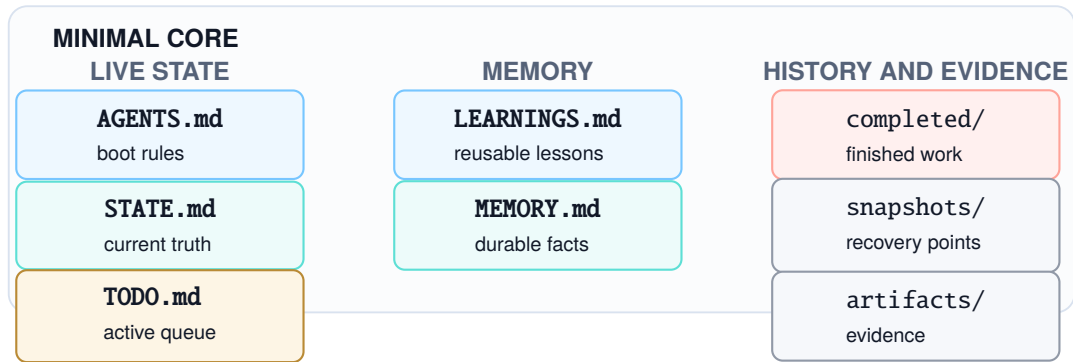


Figure 6. The minimal core separates live state from archival material. Upper-case filenames denote the live state files.

3.3 State, history, and knowledge are different objects

Current state is overwritten. History accumulates. Knowledge is distilled. The operational grammar depends on keeping those three objects separate.

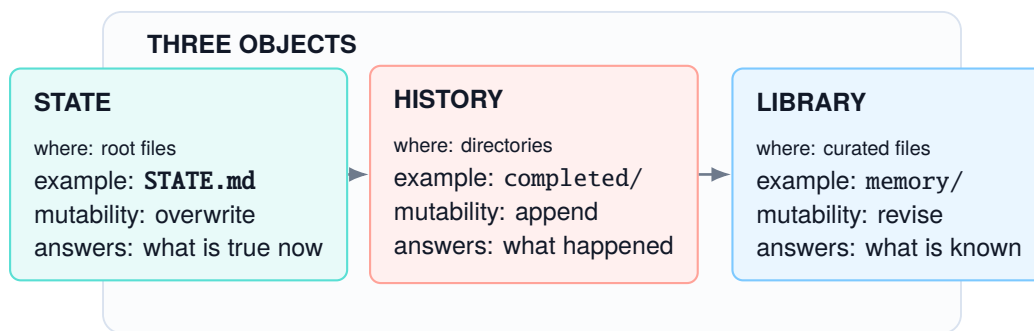


Figure 7. Current truth, history, and reusable knowledge should not collapse into one file.

4 Operational Model

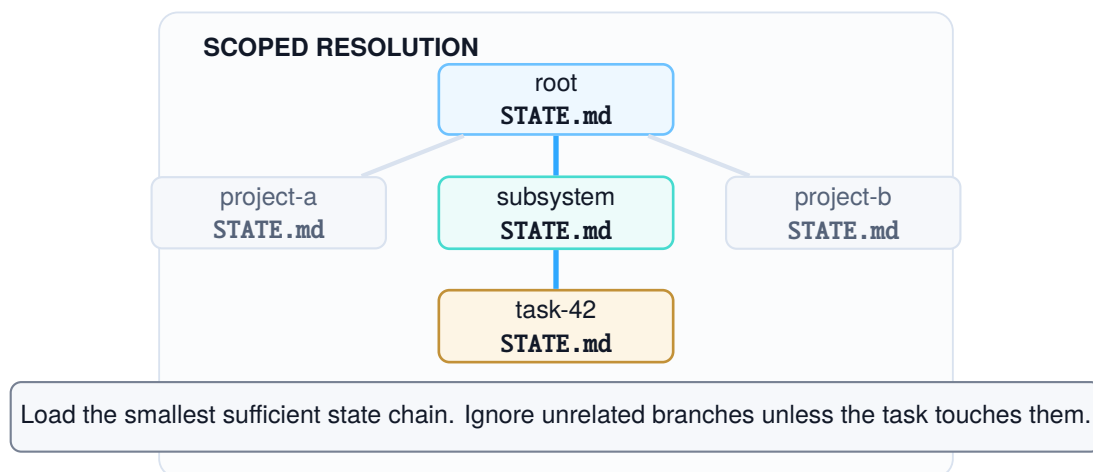


Figure 8. Scoped resolution follows the active branch and leaves unrelated repository state cold.

A Continuity workspace can be modeled as a tuple

$$C = \langle M, S, T, L, H, A, G, V, \Omega \rangle, \quad (1)$$

where M is the manifest set, S the current-state set, T the task queue, L learned memory, H semantic history, A generated artifacts, G version history, V validators and enforcement mechanisms, and Ω the active agent set.

For a path p and context budget B , Continuity does not load every available file. It loads the smallest relevant slice:

$$\text{load}(p, B) = \text{select}(M \cup S \cup T \cup L \mid \text{scope}(p)) \quad \text{s.t.} \quad \text{tokens}(\text{load}) \leq B. \quad (2)$$

The function is intentionally abstract. Its purpose is normative, not implementation-specific: keep the live context small, task-specific, and explicit.

4.1 Reconciliation invariant

Every action that changes repository reality risks invalidating recorded state. Continuity therefore requires a reconciliation step after meaningful work:

$$\text{state}(C_{\text{after}}) \approx \text{reality}(R_{\text{after}}). \quad (3)$$

The approximation symbol is deliberate. Natural-language state files cannot perfectly encode repository truth. They can, however, remain close enough to prevent stale planning and repeated errors.

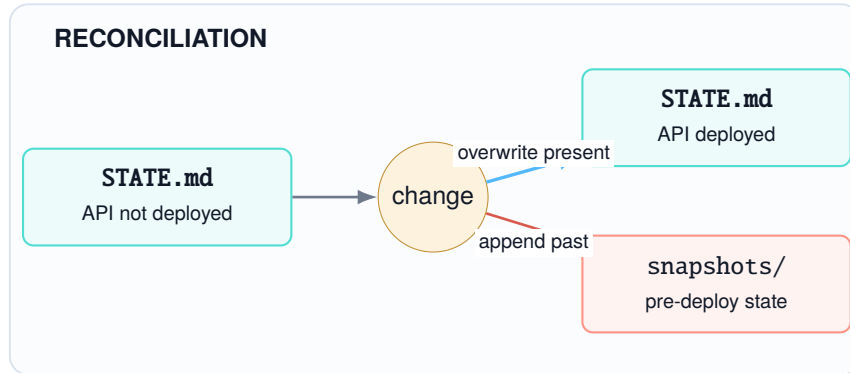


Figure 9. The live state updates in place, while the old state moves to append-only history.

4.2 Safety inheritance

Local context should refine parent guidance, not weaken it. If a repository-level file forbids destructive commands without approval, a subdirectory file must not silently remove that rule. Continuity expresses that constraint as

$$\text{safety}(\text{child}) \supseteq \text{safety}(\text{parent}). \quad (4)$$

4.3 Evidence requirement

Completion without evidence is only an assertion. Continuity therefore treats a task as complete only when a reviewable artifact exists:

$$\text{status}(\text{task}) = \text{complete} \Rightarrow \exists e \in (A \cup G \cup H) : \text{supports}(e, \text{task}). \quad (5)$$

A passing test log, an artifact bundle, a review record, or a commit can satisfy this condition.

5 Negative Evidence and Design Constraints

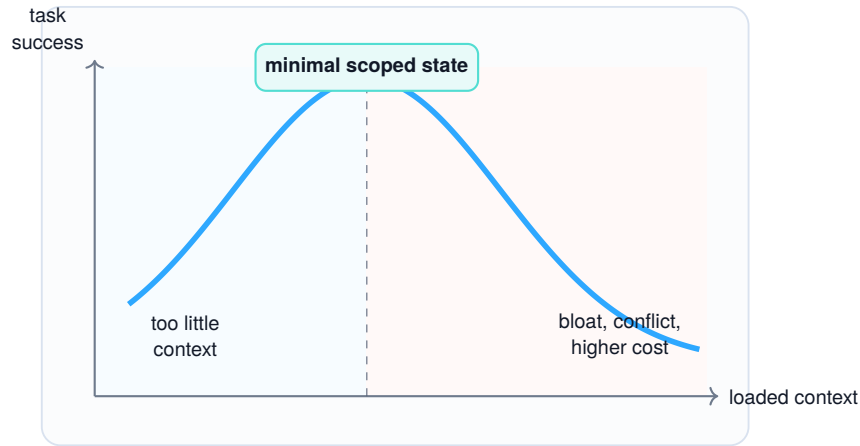


Figure 10. The design hypothesis is an inverted U: some state helps, but unscoped context becomes harmful.

The most useful recent papers in this area are cautionary. Gloaguen et al. show that repository-level context files do not automatically improve coding-agent performance and can increase inference cost substantially [6]. Dos Santos et al. show that context files themselves suffer from recurring maintenance defects [7]. CONTINUITY treats those findings as hard design pressure, not as footnotes.

Table 2. Recent negative evidence translated into design constraints.

Observed risk	Practical meaning	CONTINUITY response
Context bloat	too much low-signal text enters the live prompt	root budgets, scoped loading, archives off the hot path
Conflicting instructions	multiple files tell the agent incompatible things	lints, review, parent-child safety checks
Stale state	current files lag behind repository reality	post-action reconciliation and freshness checks
False completion	tasks marked done without proof	artifact requirement and review links
Soft policy failure	prose alone cannot enforce critical rules	hooks, tests, permissions, sandboxing
Generated noise	LLM-written summaries become canonical too early	promotion only after review
Multi-agent collision	overlapping edits create duplicate or inconsistent work	claims, branch isolation, merge checks

5.1 Anti-patterns

Manifest monolith. One giant file tries to store policy, current state, old history, and tutorial prose. The result is noise.

State diary. `STATE.md` becomes a chronological log instead of a compact statement of current truth.

Archive-on-boot. Old snapshots and completed work load by default, wasting context budget.

False completion. Tasks move to done without tests, artifacts, or reviewable evidence.

Unbounded local override. A nested file quietly weakens a repository-level safety rule.

Each anti-pattern has the same cure: keep state narrow, move history out of the hot path, and enforce what prose alone cannot enforce.

6 Implementation Blueprint

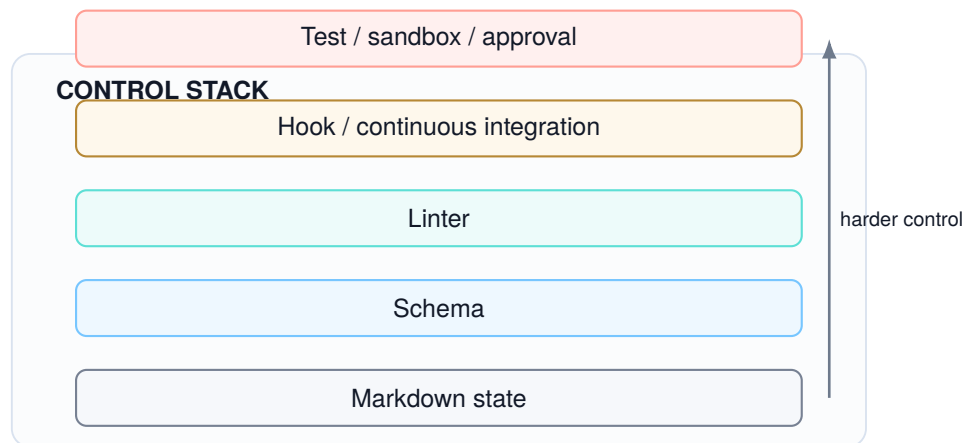


Figure 11. Readable text is useful. Reliable control requires stronger layers around it.

A practical Continuity implementation has two layers. The first layer is human-readable Markdown or YAML front matter. The second layer is deterministic validation.

6.1 Example file header

```
---
continuity_type: state
scope: repository
loads_on: boot
mutability: overwrite-current
updates_after:
  - architecture_change
  - task_completion
  - failed_assumption
validation:
  - max_lines: 180
  - no_unresolved_placeholders
  - no_conflicting_status
---
```

6.2 Hooks and validators

A useful implementation should provide at least five checks.

1. **Pre-load budget checks** stop archive files or oversized manifests from loading by default.
2. **Pre-action scope checks** verify that the agent has loaded the nearest relevant state chain.

3. **Post-action reconciliation checks** flag when code changed but **STATE.md** or **TODO.md** did not.
4. **Artifact checks** reject task completion without evidence links, test output, or commits.
5. **Safety checks** detect nested files that weaken parent safety rules.

6.3 Promotion rules

Generated state should not become canonical automatically. A run summary written by an agent can seed **LEARNINGS.md** or **completed/**, but promotion into **STATE.md** or repository-wide memory should pass through review, linting, and freshness checks.

6.4 Why this layer matters

The implementation layer prevents the central category error of the whole topic: confusing readable text with reliable control. Continuity is strongest when the repository shows the state and the surrounding tooling checks that the state still makes sense.

7 Multi-Agent Collaboration

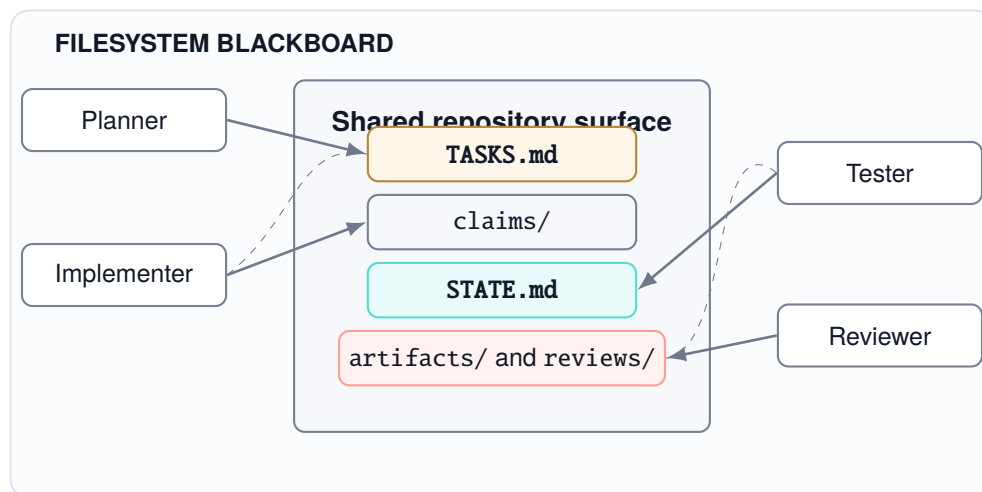


Figure 12. Agents coordinate through a shared file surface. Locks, permissions, and scheduling still live outside the files themselves.

CONTINUITY becomes more useful when several agents or humans share the same repository, but multi-agent use also raises the cost of ambiguity. A shared **TASKS.md** is helpful only if agents can claim work, avoid overlap, and leave handoff evidence behind.

7.1 Repository as filesystem blackboard

The shared repository can play the role of a versioned blackboard. Agents read the active queue, claim a task, load the relevant state slice, execute work on a branch, publish evidence, and request review. That pattern aligns with blackboard coordination and contract-based allocation without pretending that the repository itself is a scheduler [23, 24].

7.2 Minimal collaboration artifacts

A workable multi-agent setup needs only a few extra artifacts:

- **TASKS.md** for the queue,
- `claims/` for task ownership and expiry,
- `BINDING.schema.json` for role or permission contracts,
- `artifacts/` for outputs and test evidence,
- `reviews/` for approval and merge decisions.

7.3 Collision control

Two agents should not edit the same semantic object without coordination. In practice that means branch isolation, expiring claims, merge checks, and review gates. CONTINUITY can record these controls, but external tooling must enforce them.

7.4 Handoff quality

The main win is not full autonomy. The main win is cleaner handoff. When an implementer agent stops, the tester or reviewer should be able to answer three questions immediately: what changed, what remains, and what proves the current state.

8 Evaluation Agenda



Figure 13. The evaluation should compare useful state, no state, and harmful state.

The right evaluation target is not "does any context file help?" The right target is narrower: under what conditions does a compact, validated state layer improve repository work, and when does it become overhead?

8.1 Conditions

The core conditions are baseline (no repository state), manifest-only, minimal Continuity, minimal Continuity with hooks, full topology, and failure modes such as stale state, bloated context, and conflicting local instructions.

8.2 Benchmarks and tasks

Existing repository benchmarks such as SWE-bench remain useful for patch correctness [38]. They do not, however, measure resumption, repeated-error avoidance, or multi-agent collision. CONTINUITY therefore needs a custom task family with interruptions, stale-state traps, cross-branch handoffs, and malicious or noisy context.

8.3 Metrics

Table 3. Primary metric families for a CONTINUITY evaluation.

Metric family	Examples
Task outcome	success rate, pass@1, regression count, patch correctness
Efficiency	input tokens, output tokens, tool calls, wall-clock time, cost
Continuity	resumption latency, repeated-error rate, stale-state incidence, reconciliation completeness
Multi-agent	claim conflict rate, duplicate work, merge conflicts, handoff success
Governance	instruction compliance, review time, smell prevalence, override frequency
Security	prompt-injection obedience, secret leakage, unauthorized action attempts

The most interesting metrics are often not the headline ones. Task success matters, but so do resumption latency, repeated-error rate, evidence completeness, human review time, and collision frequency.

8.4 Ablations

Ablations should remove **STATE.md**, **TODO.md**, **LEARNINGS.md**, scoped loading, evidence requirements, and validation hooks. Additional ablations should intentionally inject context bloat, stale facts, and local instruction conflicts.

8.5 Expected shape of the results

The expected positive result is modest: faster restarts, better handoff, cleaner audit trails, and fewer repeated failures. The expected negative result is equally important: larger context topologies should cost more and may perform worse. A credible paper should publish both.

9 Threat Model

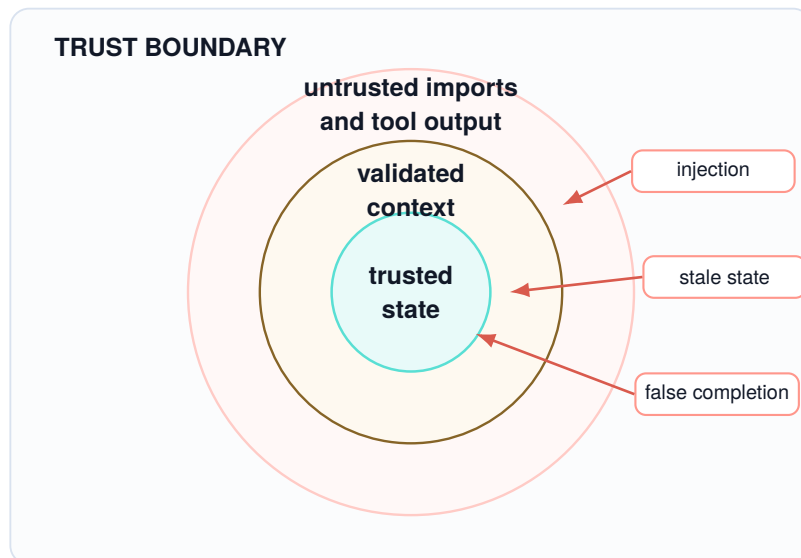


Figure 14. Trust rises only after validation. Raw repository text is not automatically safe context.

Continuity moves useful state into plain files. That improves visibility, but it also creates an obvious attack surface.

9.1 Assets

The protected assets are source code, secrets, project state, task queues, branch and review records, generated artifacts, and tool permissions.

9.2 Adversaries

Relevant adversaries include malicious pull requests, poisoned issue bodies, compromised documentation, prompt-injected tool output, careless agents, stale generated summaries, and over-permissioned automation.

9.3 Failure modes

The core failures are prompt injection, silent instruction override, stale-state obedience, false completion, secret persistence, and agent collision. Multi-agent use adds lock poisoning, fake claims, and adversarial handoff records.

9.4 Mitigations

Table 4. Condensed threat model.

Element	Examples
Assets	source code, secrets, state files, task queues, artifacts, permissions
Adversaries	malicious pull requests, poisoned issues, prompt-injected docs, careless agents
Attack surfaces	manifests, state files, imported text, tool output, generated summaries, hooks
Failures	injection, stale-state obedience, false completion, secret persistence, agent collision
Mitigations	validation, context budgets, hooks, sandboxing, least privilege, review, signed commits

The mitigation stack is straightforward in principle: treat imported text as untrusted, validate every trusted file, require evidence for completion, keep permissions narrow, and insist on review before high-impact state changes become canonical.

10 Discussion and Limits

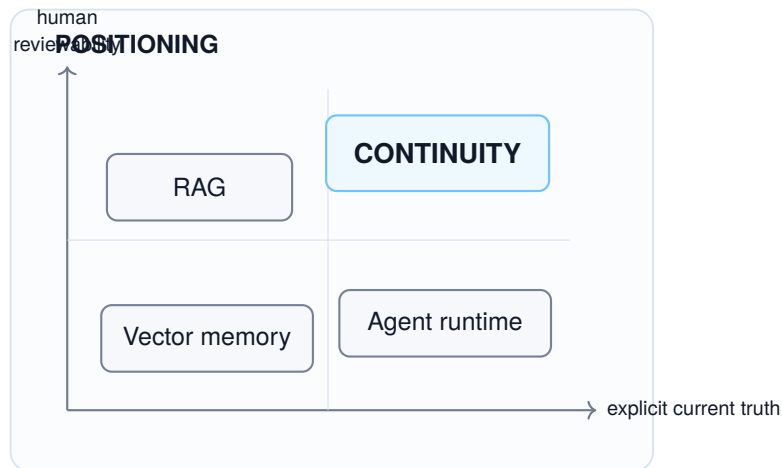


Figure 15. CONTINUITY occupies a narrower niche than retrieval and orchestration layers.

10.1 Why Markdown remains the default

Markdown is still the right starting substrate because it is inspectable, diffable, portable, and already understood by current coding agents. It also fails gracefully: when a file is wrong, a human can repair it with a text editor.

10.2 Why Continuity is not just retrieval-augmented generation or a memory database

retrieval-augmented generation retrieves possibly relevant material. CONTINUITY records mutable operational truth: what is currently true, what is next, and what evidence exists. Memory databases can complement that layer, but they do not remove the need for explicit state artifacts that humans can review.

10.3 Why Continuity is not an agent runtime

The repository cannot schedule workers, enforce permissions, or guarantee isolation by itself. Continuity is a control surface and a persistence convention. Runtime discipline still lives in the surrounding toolchain.

10.4 Main limitation

The design can collapse into paperwork. If the files become ceremonial, overlong, or stale, the system turns into a liability. That is why the paper repeatedly returns to one constraint: the defensible system is the smallest one that still preserves continuity.

11 Conclusion

Continuity argues for a small but durable shift in how coding agents work with repositories. The repository should not store code alone. It should also store the minimal state needed to let work survive a session boundary: current truth, active tasks, validated lessons, append-only history, and reviewable evidence.

That claim is strongest when it remains narrow. External memory research supports persistent state. Repository-local manifest practice shows that current tools already lean this way. Architecture and provenance research explain why the state should be inspectable. Security research explains why the same files cannot be

trusted blindly. The resulting design stance is clear: keep the state layer compact, scoped, validated, and easy to delete. Anything more should earn its place by evidence.

References

- [1] OpenAI. *Custom Instructions with AGENTS.md – Codex Documentation*. Accessed 2026-06-29. 2026. URL: <https://developers.openai.com/codex/guides/agents-md>.
- [2] Anthropic. *How Claude Remembers Your Project – Claude Code Documentation*. Accessed 2026-06-29. 2026. URL: <https://code.claude.com/docs/en/memory>.
- [3] Anthropic. *Automate Actions with Hooks – Claude Code Documentation*. Accessed 2026-06-29. 2026. URL: <https://code.claude.com/docs/en/hooks-guide>.
- [4] Nous Research. *Context Files | Hermes Agent*. Accessed 2026-06-29. 2026. URL: <https://hermes-agent.nousresearch.com/docs/user-guide/features/context-files>.
- [5] OpenClaw. *Memory Overview*. Accessed 2026-06-29. 2026. URL: <https://docs.openclaw.ai/concepts/memory>.
- [6] Thibaud Gloaguen, Niels M"undler, Mark M"uller, Veselin Raychev, and Martin Vechev. *Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?* 2026. arXiv: [2602.11988](https://arxiv.org/abs/2602.11988) [cs.SE]. URL: <https://arxiv.org/abs/2602.11988>.
- [7] Helio Victor F. dos Santos, Vitor Costa, Joao Eduardo Montandon, Luciana Lourdes Silva, and Marco Tulio Valente. *Configuration Smells in AGENTS.md Files: Common Mistakes in Configuring Coding Agents*. 2026. arXiv: [2606.15828](https://arxiv.org/abs/2606.15828) [cs.SE]. URL: <https://arxiv.org/abs/2606.15828>.
- [8] Charles Packer et al. *MemGPT: Towards LLMs as Operating Systems*. 2023. arXiv: [2310.08560](https://arxiv.org/abs/2310.08560) [cs.AI]. URL: <https://arxiv.org/abs/2310.08560>.
- [9] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Wang, and Yanlin Zhou. *MemoryBank: Enhancing Large Language Models with Long-Term Memory*. 2023. arXiv: [2305.10250](https://arxiv.org/abs/2305.10250) [cs.CL]. URL: <https://arxiv.org/abs/2305.10250>.
- [10] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. "Generative Agents: Interactive Simulacra of Human Behavior". In: *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 2023.
- [11] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023. arXiv: [2303.11366](https://arxiv.org/abs/2303.11366) [cs.AI]. URL: <https://arxiv.org/abs/2303.11366>.
- [12] Lei Wang et al. *A Survey on Large Language Model based Autonomous Agents*. 2023. arXiv: [2308.11432](https://arxiv.org/abs/2308.11432) [cs.AI]. URL: <https://arxiv.org/abs/2308.11432>.
- [13] Dewayne E. Perry and Alexander L. Wolf. "Foundations for the Study of Software Architecture". In: *ACM SIGSOFT Software Engineering Notes* 17.4 (1992), pp. 40–52.
- [14] Philippe Kruchten. "The 4+1 View Model of Architecture". In: *IEEE Software* 12.6 (1995), pp. 42–50.
- [15] Jeff Tyree and Art Akerman. "Architecture Decisions: Demystifying Architecture". In: *IEEE Software* 22.2 (2005), pp. 19–27.
- [16] Anton Jansen and Jan Bosch. "Software Architecture as a Set of Architectural Design Decisions". In: *Working IEEE/IFIP Conference on Software Architecture*. 2005.
- [17] ISO/IEC/IEEE. *ISO/IEC/IEEE 42010: Systems and Software Engineering – Architecture Description*. 2011.
- [18] Mohamed Soliman et al. *An Exploratory Study on Architectural Knowledge in Issue Tracking Systems*. 2021. arXiv: [2106.11140](https://arxiv.org/abs/2106.11140) [cs.SE]. URL: <https://arxiv.org/abs/2106.11140>.

- [19] Greg Wilson, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. “Good Enough Practices in Scientific Computing”. In: *PLOS Computational Biology* 13.6 (2017), e1005510.
- [20] W3C. *PROV-DM: The PROV Data Model*. 2013. URL: <https://www.w3.org/TR/prov-dm/>.
- [21] Stian Soiland-Reyes et al. “Packaging Research Artefacts with RO-Crate”. In: *Data Science* 5 (2022), pp. 97–138.
- [22] Leonardo Murta, Vanessa Braganholo, Fernando Chirigati, David Koop, and Juliana Freire. “noWorkflow: Capturing and Analyzing Provenance of Scripts”. In: *International Provenance and Annotation Workshop*. 2014.
- [23] Daniel D. Corkill. “Blackboard Systems”. In: *AI Expert* 6.9 (1991), pp. 40–47.
- [24] Reid G. Smith. “The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver”. In: *IEEE Transactions on Computers* C-29.12 (1980), pp. 1104–1113.
- [25] David Gelernter. “Generative Communication in Linda”. In: *ACM Transactions on Programming Languages and Systems* 7.1 (1985), pp. 80–112.
- [26] Carl Hewitt, Peter Bishop, and Richard Steiger. “A Universal Modular Actor Formalism for Artificial Intelligence”. In: *Proceedings of the 3rd International Joint Conference on Artificial Intelligence*. 1973.
- [27] Qingyun Wu et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. 2023. arXiv: [2308.08155 \[cs.AI\]](https://arxiv.org/abs/2308.08155). URL: <https://arxiv.org/abs/2308.08155>.
- [28] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. *CAMEL: Communicative Agents for Mind Exploration of Large Language Model Society*. 2023. arXiv: [2303.17760 \[cs.AI\]](https://arxiv.org/abs/2303.17760). URL: <https://arxiv.org/abs/2303.17760>.
- [29] Sirui Hong et al. *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework*. 2023. arXiv: [2308.00352 \[cs.AI\]](https://arxiv.org/abs/2308.00352). URL: <https://arxiv.org/abs/2308.00352>.
- [30] Edwin Hutchins. *Cognition in the Wild*. MIT Press, 1995.
- [31] David Kirsh. “The Intelligent Use of Space”. In: *Artificial Intelligence* 73.1–2 (1995), pp. 31–68.
- [32] Daniel M. Wegner. “Transactive Memory: A Contemporary Analysis of the Group Mind”. In: *Theories of Group Behavior*. Springer, 1985.
- [33] Kjeld Schmidt and Carla Simone. “Coordination Mechanisms: Towards a Conceptual Foundation of CSCW Systems Design”. In: *Proceedings of the 1996 ACM Conference on Computer Supported Cooperative Work*. 1996.
- [34] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. *Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. 2023. arXiv: [2302.12173 \[cs.CR\]](https://arxiv.org/abs/2302.12173). URL: <https://arxiv.org/abs/2302.12173>.
- [35] Yi Liu et al. *Prompt Injection Attack against LLM-Integrated Applications*. 2023. arXiv: [2306.05499 \[cs.CR\]](https://arxiv.org/abs/2306.05499). URL: <https://arxiv.org/abs/2306.05499>.
- [36] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications*. 2025. URL: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- [37] OpenSSF. *Supply-chain Levels for Software Artifacts*. 2024. URL: <https://slsa.dev/>.
- [38] Carlos E. Jimenez et al. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” In: *International Conference on Learning Representations*. 2024.

A Appendix A: Minimal file templates

Example STATE.md

```
# State
```

```
## System summary
Authentication service uses OAuth 2.1 with PKCE. Token refresh is server side.

## What changed recently
- Callback validation moved into auth middleware.
- Legacy refresh endpoint is deprecated but still present.

## Active constraints
- Do not change the token schema without migration notes.
- Any auth flow change requires integration tests.

## Next reconciliation point
Update after the current callback test suite finishes.
```

Example TODO.md

```
- [in-progress] Add callback replay protection
  owner: implementer-02
  evidence: artifacts/auth/replay-tests.txt

- [open] Remove legacy refresh endpoint after migration note lands
  dependency: callback replay protection
```

B Appendix B: Suggested lints

- Reject unresolved placeholders such as `{{VARIABLE}}` in trusted state files.
- Reject root manifests that exceed a configurable line or token budget.
- Warn when `STATE.md` contains stale timestamps or diary-style history.
- Warn when archives appear in default load lists.
- Reject nested files that weaken parent safety rules.
- Reject completed tasks that do not reference commits, tests, or artifacts.
- Warn when generated summaries are promoted without review metadata.